Difficulty Classification in Long, Heterogeneous Prompts

1. Problem Statement

Semantic routers classify incoming requests to decide which model should handle them. One important routing signal is **task complexity/difficulty**—for example, whether a request should be handled by an easy, medium, or hard model.

The problem becomes more difficult when the request contains a large amount of additional context.

For example, a coding/bug-fixing request may look like:

```
None
User task:
"Fix the null-pointer exception"

+ source code
+ stack trace
+ test cases
+ repository information
+ documentation
+ previous conversation
```

The **actual task** may be only a small portion of the complete prompt.

Therefore, a complexity classifier operating on the complete prompt can face two different issues:

1. **Long-context problem** — processing a very large input makes signal extraction more expensive.
2. **Difficulty-classification problem** — the information representing the task's difficulty may be mixed with large amounts of auxiliary context.

The important distinction is:

Prompt length/context size does not necessarily represent task difficulty.

For example:

```
Long prompt + simple task  → Easy
Short prompt + complex task → Hard
```

This creates the research problem:

> **How can a semantic router estimate the actual difficulty of a task from a long, heterogeneous prompt without allowing auxiliary context to distort the difficulty representation?**

This is particularly relevant to vLLM Semantic Router. Its current architecture separates signals such as context, structure, complexity and modality, while its Athena release introduced prompt compression specifically before signal extraction.

2. Proposed Approaches

Approach	Basic idea	Example Open-source solutions	Relevance
1. Context / Query Reduction	Reduce the amount of information given to the classifier while preserving important information	**LLMLingua**, **LongLLMLingua**	Addresses the effect of excessive context
2. LLM-based Classification	Use an LLM directly to understand the request and produce a classification/decision	**AnyJev**	Addresses classification using an LLM rather than conventional similarity
3. Task / Feature Characterization	Extract characteristics/signals from the request before making routing decisions	**vLLM Semantic Router Athena/Themis**	Addresses the problem of treating the entire prompt as one representation

Approach 1 — Context / Query Reduction

Idea

Instead of sending the complete prompt to the difficulty classifier:

```
None
Long prompt
     ↓
Remove / compress less relevant information
     ↓
Reduced representation
     ↓
Difficulty classification
```

The objective is **not simply to reduce tokens**. The important requirement is to preserve information that is relevant to the downstream task.

LLMLingua - LLMLingua GitHub

LLMLingua is Microsoft's open-source prompt-compression framework. It uses a smaller language model to identify and remove non-essential tokens from prompts. The project includes LLMLingua, LongLLMLingua and LLMLingua-2.

LongLLMLingua - LongLLMLingua paper

specifically targets long-context scenarios and uses question-aware compression to preserve information relevant to the question.

Important limitation for your problem:

Compression solves:

> "There is too much context."

But it does not automatically solve:

> "Which remaining information actually determines task difficulty?"

So it is relevant, but not a complete solution to your specific problem.

Approach 2 — LLM-based Classification

Idea

Instead of using a conventional embedding/similarity mechanism to classify the request:

```
None
Prompt
   ↓
Embedding
   ↓
Similarity
   ↓
Difficulty
```

an LLM itself can be asked to make a structured decision:

```
None
Prompt
   ↓
LLM
   ↓
Structured decision
```

For example:

```
None
Question:
How difficult is this coding task?

Options:
Easy / Medium / Hard
```

AnyJev - AnyJev decision documentation

AnyJev, from Nokia Applied Research, is an open-source framework for turning an LLM into a typed decision model. It supports decisions such as:

- choice among categories
- Yes/No decisions
- ordinal scores

It supports Transformers and vLLM backends.

For example, its documentation demonstrates decisions such as:

> Which handler should process this request?

with outputs such as:

```
billing
technical
sales
other
```

and associated probabilities.

AnyJev also provides different decision levels, including direct logit-based decisions and hidden-state-based decision heads.

Important limitation for your research:

AnyJev is a **general-purpose LLM decision/classification framework**. It is not specifically designed to solve the problem of difficulty classification being distorted by long coding context.

AnyJev GitHub

Approach 3 — Task / Feature Characterization

Idea

Instead of treating the entire prompt as one representation:

```
Full prompt
      ↓
Easy / Medium / Hard
```

the system first extracts **different characteristics of the request**:

```
Full prompt
      ↓
Task characteristics / signals
      ↓
Difficulty-related information
      ↓
Routing decision
```

For a coding request, the characteristics could describe things such as:

```
Task type       → bug fixing
Error present   → yes
Code present    → yes
Conversation    → multi-turn
Context size    → large
Structure       → code-heavy
```

vLLM Semantic Router — Athena/Themis - [vLLM Semantic Router v0.3 Themis documentation/blog]

This is where vLLM Semantic Router is particularly relevant.

Themis explicitly separates the routing pipeline into:

```
Signal
   ↓
Projection
   ↓
Decision
   ↓
Algorithm
   ↓
Model
```

Signals extract evidence from the request, including information related to context, complexity, conversation, structure, modality and other properties.

The current implementation also dispatches **context, structure, complexity and modality as separate classifier signals**, rather than treating them as one identical signal.

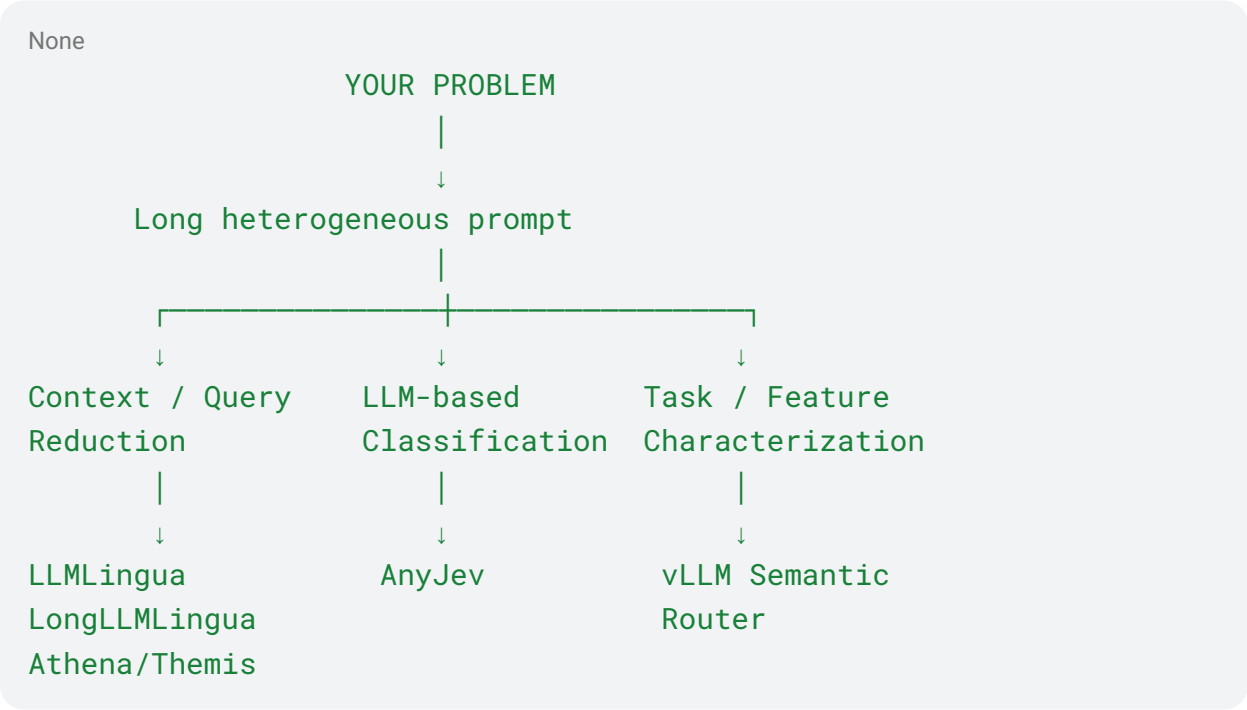
This makes vLLM Semantic Router an important existing example for studying **task characterization through multiple signals**.

Open source: Yes.

[vLLM Semantic Router GitHub]

3. Summary of the Literature Space

Your literature survey can therefore be represented as:



The key distinction

Approach	Main question it answers
Context / Query Reduction	*What information can we remove while retaining useful information?*
LLM-based Classification	*Can an LLM directly understand and classify the request?*

Task / Feature Characterization	*What characteristics of the request should be extracted before making the routing decision?*